arXiv:2312.09241v1 [cs.LG] 14 Dec 2023

TinyGSM: achieving > 80% on GSM8k with small language models

Bingbin Liu[1], Sebastien Bubeck[2], Ronen Eldan[2], Janardhan Kulkarni[2],
Yuanzhi Li[2], Anh Nguyen[2], Rachel Ward[2], Yi Zhang[2]

[1]Carnegie Mellon University
[2]Microsoft Research

Abstract

Small-scale models offer various computational advantages, and yet to which extent size is critical for problem-solving abilities remains an open question. Specifically for solving grade school math, the smallest model size so far required to break the 80% barrier on the GSM8K benchmark remains to be 34B. Our work studies how high-quality datasets may be the key for small language models to acquire mathematical reasoning. We introduce TinyGSM, a synthetic dataset of 12.3M grade school math problems paired with Python solutions, generated fully by GPT-3.5. After finetuning on TinyGSM, we find that a duo of a 1.3B generation model and a 1.3B verifier model can achieve 81.5% accuracy, outperforming existing models that are orders of magnitude larger. This also rivals the performance of the GPT-3.5 "teacher" model (77.4%), from which our model's training data is generated. Our approach is simple and has two key components: 1) the high-quality dataset TinyGSM, 2) the use of a verifier, which selects the final outputs from multiple candidate generations.

1 Introduction

One fascinating phenomenon regarding large language models (LLMs) is the emergence of capbilities as both the model and dataset sizes scale up (Wei et al., 2022a; Chan et al., 2022). Among many capabilities, mathematical reasoning is one particular aspect that has received tremendous attention Lewkowycz et al. (2022); Lightman et al. (2023). However, it is unclear to what extend scale is a necessity for mathematical reasoning, and the potential of small language models (SLMs) remains largely under-explored.

In this work, we push the boundaries of SLMs' math reasoning capabilities. As a first step towards general mathematics, our primary testing ground is grade school math problems, to solve which require both mathematical understanding and language comprehension. The gold-standard benchmark in this regime is GSM8K (Cobbe et al., 2021), a collection of 8.8K grade-school math word problems (with a 7k-1k train-test split) that involve 2 to 11 reasoning steps. GSM8K has been widely regarded to be challenging for LLMs. Even though the questions appear rather simple for humans, there have been few models that achieve > 80%, and they are commonly of prohibitive sizes, e.g. 34B and above (see Table 1).

Our goal is to break the 80% barrier on GSM8K while keeping the model size friendly. As previous work shows (Gunasekar et al., 2023; Li et al., 2023; Eldan & Li, 2023), training data quality is one of the most important factors for enhancing performance of small models. In particular, prompt-engineered synthetic data generation from gigantic models such as GPT-3.5/4 enjoys the clear advantage of desirable data hygiene and controllable diversity. This constituents a teacher-student scenario where the student learns from teacher's generations. On the tasks that the model model already excels at, their guided generations remain one of the highest quality data one can collect for training significantly smaller student models. It is also understood that the student model's performance likely ends up inferior than the teacher, and may fall far short especially when the student is considerably smaller than the teacher (Mirzadeh et al., 2019; Gudibande et al., 2023) —after all, the teacher places an information-theoretic bottleneck on the student.

To our surprise, in the case of GSM8K, we are able to bridge the performance gap between the student and teacher, by utilizing a tiny amount of labeled *real* data (the original GSM8K training set of 7k questions) to train an independent verifier model. At test time, the verifier score and select among multiple candidate answers generated from the student, and then we output the highest score generation as the final submission. Note the idea of using

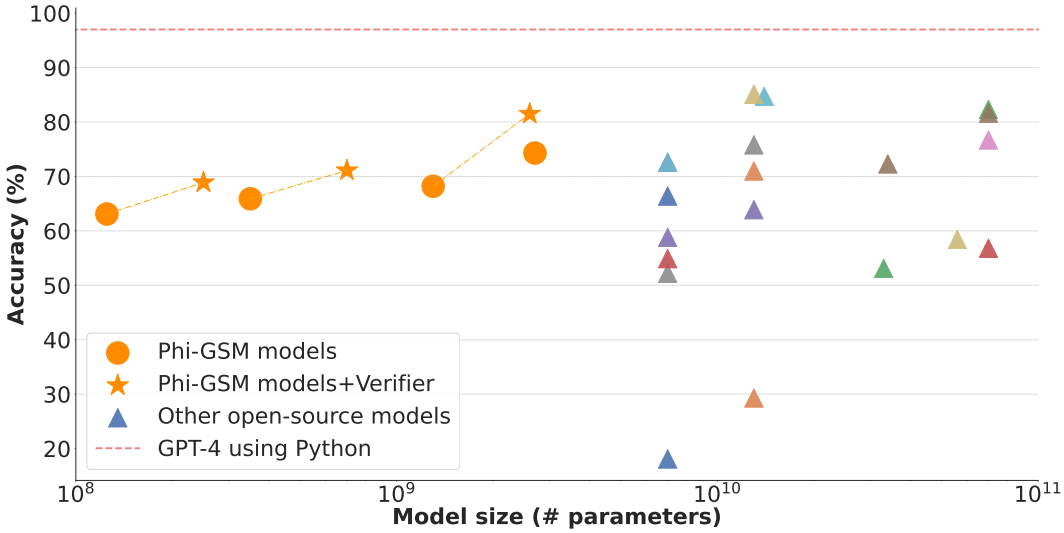

Figure 1: Our results on GSM8K. Please refer to Table 1 for details.

a verifier is proposed by the seminal GSM8K paper (Cobbe et al., 2021), and here we demonstrate its power of bridging the teacher-student gap, and we conduct a more thorough examination of factors affecting its efficacy.

The contributions of our work are the following:

- We introduce `TinyGSM`, a synthetic dataset containing GSM8K-style math word problems paired with Python solutions, generated fully by GPT-3.5-turbo. TinyGSM consists of 12.3M questions which amount to 1.8B tokens. We demonstrate `TinyGSM`'s high-quality by finetuning the Phi-1.5 1.3B model (before the use of verifiers) which improves its accuracy from 44.6% to **68.2%** on the GSM8K test set. Notably, our smallest 125M model can also achieve **63.1%** after finetuning on `TinyGSM`.

- We demonstrate the power of *verifiers* on small-scale models. When integrated with a verifier for scoring generations, our models, named Phi-GSM models, achieve performance on par with other open source models that are orders of magnitude larger. In particular, our 1.3B model achieves 81.5% accuracy on GSM8K, as shown in Figure 1. This marks a new state-of-the-arts on billion-parameter-scale models, significantly outperforming existing open-source models and even rivaling the 77.4% accuracy of GPT-3.5, from which `TinyGSM` is generated. For verifier training, we identify *data diversity* as a crucial element for a verifier's success, and find that the *scaling of the verifier* may be more effective than scaling of the generator: while scaling up from a 125M generator to a 1.3B generator only gives a 5.1% increase in performance (Table 1), scaling up the verifier from 125M to 1.3B leads to a 7.2% performance boost (Figure 4).

2 Related works

Distilling from synthetic data: While scaling up has been a useful strategy, it is possible to outpace conventional scaling laws by better use of data (Sorscher et al., 2022). In the data-scarce case, quality synthetic data serves as an effective workaround (Eldan & Li, 2023; Gunasekar et al., 2023), and the scaling in dataset size can compensate for a small model size (Edelman et al., 2023). Additionally, our work uses samples in the true distribution (i.e. the GSM8K train set) differently: given the small dataset size, we believe that the most sample-efficient way to utilize the true train set is to train a verifier—while the 7.4k samples in the GSM8K training set is too small for language model finetuning, it is sufficient for training a good quality verifier that provides 10% performance boost. While there have been potential concerns of learning from synthetic data such as loosing diversity or having a drifted distribution mean (Alemohammad et al., 2023; Shumailov et al., 2023), Alemohammad et al. (2023) showed that such degradation can be avoided by including fresh samples from the true distribution during training.

Model	Base model	Model size	Answer format	Eval method	GSM8K (%)
Llama-2 (Touvron et al., 2023)	-	7B	nlp	pass@1	14.6
		13B			28.7
		34B			42.2
		70B			56.8
MetaMath (Yu et al., 2023b)	Llama-2	7B	nlp	pass@1	66.5
		13B			72.3
		70B			**82.3**
WizardMath (Luo et al., 2023)	Llama-2	7B	nlp	pass@1	54.9
		13B			63.9
		70B			**81.6**
MAmmoTH (Yue et al., 2023)	Code-Llama	7B	code	pass@1	59.4
	Code-Llama	12B			64.7
	Code-Llama	34B			72.7
	Llama-2	70B	nlp		76.9
Mistral (Jiang et al., 2023)	-	7B	nlp	maj1@8	52.2
		8×7B			58.4
OVM (Yu et al., 2023a)	Llama-2	7B+7B	nlp	verify100@1	73.7
	Mistral	7B+7B			**84.7**
Llemma (Azerbayev et al., 2023)	Llama-2	7B	nlp	pass@1	36.4
		34B			51.5
ToRA-Code (Gou et al., 2023)	Llama-2	7B	code	COT@1	72.6
		13B			75.8
		34B			**80.7**
		70B			**84.3**
Orca 2 (Mitra et al., 2023)	Llama-2	7B	nlp	pass@1	55.72
		13B			65.73
Gemini Pro	-	-	nlp	maj1@32	**86.5**
Gemini Ultra (Gemini Team)					94.4
GPT-3.5-0613	-	-	code	pass@1	77.4*
GPT-4-0613 (OpenAI, 2023)					**97.0***
Phi-1.5 (Li et al., 2023)	-	1.3B	code	pass@1	44.6
Phi-GSM	Phi-1.5-tiny	125M	code	pass@1	63.1
	Phi-1.5-small	350M			65.9
	Phi-1.5	1.3B			68.2
	Phi-2	2.7B			74.3
Phi-GSM+V	Phi-1.5-tiny	125M+125M	code	verify48@1	68.9
	Phi-1.5-small	350M+350M			71.3
	Phi-1.5	1.3B+1.3B			**81.5**

Table 1: Results on GSM8K. * denotes results measured by ourselves. Accuracies **above 80%** are labeled in **bold**. '8×7B' stands for mixture of 8 experts, and each expert is of 7B parameters. '7B+7B' means a combination of a 7B generation model plus a 7B verifier model. '+V' denotes the use of verifier models.

Math word problem datasets GSM8K (Cobbe et al., 2021) has been the most common used math word problem dataset for its quality and size. In comparison, earlier datasets such as MAWPS (Koncel-Kedziorski et al., 2016), ASDiv (Miao et al., 2020) and SVAMP (Patel et al., 2021) are either much smaller in size or of less difficulty. However, GSM8K questions are too clean to test for robustness. Motivated by the observation that language models are not robust to the presence of irrelevant context, Shi et al. (2023a) proposed GSM-IC (irrelevant context). Another problem is the GSM8K dataset itself is still too small for training language models. (Ni et al., 2023b) addressed this with self-sampled data. In a work concurrent to ours, Yu et al. (2023b) bootstraps an original dataset using various augmentation techniques, such as generating multiple answers for the solution, question rephrasing, and backward reasoning. The proposed MetaMath dataset consists of 40000 questions from GSM8K and MATH (Hendrycks et al., 2021). In comparison, TinyGSM is significantly larger, encompassing 12.3M questions (or equivalently 1.8B tokens).

Leveraging multiple generations: An important component of our method is to leverage multiple generation. This idea has been proven successful in many prior works. A notable example is "self-consistency" (Wang et al., 2022), which selects the most frequent response among candidates and integrates well with other methods such as progressive-hint prompting (Zheng et al., 2023) and model selection (Zhao et al., 2023). However, self-consistency was not particularly helpful in our setup as mentioned in Section 4.2. More related to and a direct inspiration of our work is Cobbe et al. (2021), which uses a verifier to select the best response among 100 candidates, leading to an 20% accuracy boost. Our work conducts a more thorough study on the design choices of the verifier, including data diversity and the effect of verifier sizes. Another design choice orthogonal to ours is the supervision signals, such as outcome-based supervision versus process supervision (Lightman et al., 2023).

Learning from partial or process supervision: In our experiments, we evaluate on the final accuracy only but train on full programs. Prior work has studied the effect of process versus outcome based supervision. Process-based supervision is shown to be particularly helpful for complex math problems (Lightman et al., 2023), though for general problems one needs to consider a cost-efficacy tradeoff (Uesato et al., 2022). When process supervision is not available, Ni et al. (2023b) proposed to learn from "self-sampled" solutions, which allows the model to learn from partially correct self-generated solutions selected based on the execution trace.

Self-improvement: Several works have explored the idea of "self-improvement" where a model evaluates and corrects its own generations, mostly relying on the self-debugging ability of GPT4. Examples include "self-refine" (Madaan et al., 2023) and "self-verify" (Weng et al., 2022; Zhou et al., 2023), both of which ask the model to iteratively provide feedback or verifications on its own generations and refine if needed. However, such self-improvement abilities have not been discovered in small language models. This motivated our use of a separate verifier model, which is initialized from the generative model but needs to be fully finetuned for verification.

Prompt-based methods: *Prompt-based* methods, which find prompts to improve the later conditional generations, are particularly effective for large models. Examples include *in-context learning* (Brown et al., 2020), where the model learns to perform novel tasks from few-shot examples provided in the prompt, as well as *Chain-of-Thought* (Wei et al., 2022b), which shows that explicitly generating intermediate steps can significantly help with reasoning abilities. However, similar to self-improvements, prompting is targeted at large language models and do not apply for SLMs.

3 The TinyGSM dataset

Our objective is to assess the capability of a *small* language model (SLM) on mathematical reasoning. Ideally, enhancing this mathematical reasoning ability should not compromise the model's competence in language comprehension. This makes *math word problems*, which necessitate both mathematical and language understanding, a suitable test ground. We focus on the GSM8K dataset (Cobbe et al., 2021), consisting of around 8k grade-school math word problems. The math concepts in the dataset are elementary and within standard grade-school curricula, but the challenges posed by the natural language problem statement introduce an additional layer of complexity to the task.

`TinyGSM`: augmenting GSM8K with synthetic generations Despite the high quality, the GSM8K training set only contains 7473 problems, which is too small for training a reasonably sized language model (Ni et al., 2023a). To alleviate the size issue, we augment the GSM8K training set using GPT-3.5-turbo generated synthetic problems.

We prompt GPT-3.5-turbo to generate problem variants similar to a given question (but not the solution) randomly

```python
def simple_math_problem() -> int:
    """
    In preparation for her party, Sarah buys 10 trays
    of food and 8 cases of beverages.
    Each tray costs $50 and each case of beverages
    costs $20.
    What is the total cost of the trays and beverages?
    """
    trays = 10
    tray_cost = 50
    cases = 8
    case_cost = 20
    tray_total = trays * tray_cost
    case_total = cases * case_cost
    total_cost = tray_total + case_total
    result = total_cost
    return result
```

```python
def simple_math_problem() -> int:
    """
    Kim has 4 times the number of crayons
    than 8 less than the number of markers
     she has.
    If she has 60 crayons, how many markers
     does she have?
    """
    number_crayons = 60
    number_markers = number_crayons // 4 +
    8
    result = number_markers

    return result
```

Figure 2: Examples from TinyGSM. The question is given as the docstring of a function, and the solution is the code in the function body.

sampled from the GSM8K training set. Each problem variant contains both a question and the corresponding solution written in Python, as shown in Figure 2.[1] Using code allows us to leverage a Python interpreter, circumventing language models' known limitation regarding numerical calculations and code execution.

To enhance robustness, we also generated synthetic problems whose questions contain irrelevant information. This is achieved by augmenting the GSM-IC dataset (Shi et al., 2023a), which is an augmentation of GSM8K specifically designed to introduce irrelevant context (IC) to the question statement. These GSM-IC variants constitute to approximately one third of TinyGSM.

The resulting synthetic dataset contains 12.3M problems (i.e. question-solution pairs) [2] with, based on the original 7.4k training set questions and their IC variants. For each question in the GSM8K train set, the prompt based on this question is shared across API calls, and the source of randomness comes entirely from the generation process. To encourage diversity, we use temperature sampling and specify in the prompt to encourage the problem variants to be grammatically diverse and contain multiple steps; the exact prompts are provided in Figure 3 and in Appendix A.1.

Filtering To ensure the quality of the synthetic data in `TinyGSM`, we filter out problems that are too short or do not contain numbers, as well as code solutions which are not executable. Note that we do not check for the *correctness* of the question or the generated solutions, since the "ground truth" solution is not available. Given the effectiveness of self-consistency (Wang et al., 2022), one might want to filter the problems by keeping the ones which have majority vote only. We did not adopt this strategy since we find that GPT-3.5-turbo's generations are only consistent on easy problems [3], hence such consistency filtering will remove challenging problems, resulting in a dataset that is too easy to be useful.

4 Solving grade school math with small language models

The 1.3B version of our phi-GSM models is able to achieve 81.5% accuracy on GSM8K, a dataset that remains challenging for small-scale models. The performance comes from sufficient good quality synthetic data and the use

[1] Note that the generated problems may be mathematically valid yet violating common sense. For example, some quantities may not be integers.

[2] This corresponds to 1.8B tokens, which costs around $3600 to generate according to GPT commercial pricing.

[3] "Easy" problems refer to the ones for which a 350M model, trained on a part of TinyGSM, already produces same final answer as GPT-3.5-turbo. For example, for an early version of our 350M model, the model only achieves around 50% on the GSM8K test set, but can achieve more than 87% on synthetic questions with consistent answers. In other words, adding more easy problems like these will not help our 350M model bridge the performance gap between itself and GPT-3.5-turbo.

```
Consider the following grade-school math problem: {{question}}

Generate 10 different math problems similar to this math problem.

- Make sure each question uses diverse NLP and includes multiple logical steps.
- After each generated problem, write down a **detailed and complete Python program** to solve the question **step
 by step** (do NOT give the result directly, **DO NOT write calculations in the comments**).
- The program should contain multiple lines of code and end with 'result = XXX' (Make sure to replace XXX with the
 actual result of the python program).
- Make sure your Python program is complete and solves the problem. Do **NOT** write things like 'solution to be
completed', result = ?, insert your code here etc.
- Give the complete solution to solve the problem, written in Python. Do not write things like 'insert your code
here'.
- In each new question, **first end with <|endofquestion|>**, and then start writing the program. Each program
should end with <|endofprogram|>.
- Example format: Question X: New question (at least 4 sentences long and use diverse NLP) (without the solution)
<|endofquestion|> Complete python code with entire solutions and the correct indent (<|endofprogram|>])
```

Figure 3: The prompt template for generating `TinyGSM`.

of a verifier, which we describe in this section.

4.1 Learning from synthetic data

We finetune the Phi-1.5 125M, 350M and 1.3B models on our `TinyGSM` from Section 3, and in particular, the 1.3B model reaches **68.2%** accuracy.[4] [5] We use the Adam optimizer with FP16 during training, with a linear warm-up and a maximum learning rate of 1e-4, a weight decay of 0.01, and an effective batch size of 1024. The finetuning phase takes up to 20k steps in total. As shown in Figure 1, even without verifiers, our models are already competitive to models of size from 7B and larger. As an anecdote, an earlier and worse performing version of our Phi-GSM 1.3B model gets 94% (or 82.5% from 350M at pass@32, whereas the 750M CodeT5+ model (Wang et al., 2023) gets 73.8% (or 70.5% from 220M) at pass@100.

4.2 Improving small models with a verifier

While sufficient synthetic data can significantly boost model performance, the performance is still below 70%. Does further improvement necessitate larger model and more data then? There may be two concerns: First, there may be a diminishing return in adding extra parameters and data; for instance, while there is a 10% increase in performance when increasing from around one third of the final size of TinyGSM to two thirds, the final one third of the data provided only marginal gain. Moreover, even if the small language model is able to fully match the quality of the synthetic data, GPT-3.5-turbo itself can only achieves 77.4% test accuracy on GSM8K, which seemingly poses a limit on the performance of any models distilling from its generations.

In this section, we show that the use of a verifier can be an effective strategy orthogonal to introducing more and better data, and can even help SLMs exceed the accuracy of GPT-3.5-turbo generations. The main observation that the best of multiple generations significantly outperforms a single generation. These generations could be low-temperature generations from different checkpoints of a single run, where taking the best out of generations from 5 checkpoints of (an early version of) our 350M model reaches 75% accuracy, similar to findings in temporal ensembling (Laine & Aila, 2016) and snapshot ensembles (Huang et al., 2017). [6] The generations could also be from high-temperature generations based on a single checkpoint; for instance, the pass@32 accuracy of our 1.3B model is 94%.

This suggests a promising direction of leveraging multiple generations: we can obtain a great performance boost if we are able to identify the best generation. This idea is effective yet natural: The probabilistic nature of the

[4] The Phi-1.5-small 350M and Phi-1.5-125M variants are pretrained on the same pretraining data as the Phi-1.5 1.3B model.

[5] Performance of training on `TinyGSM` from scratch is reported in Table 2.

[6] For utilizing multiple checkpoints, an option is to use model soup (Wortsman et al., 2022); however, a uniform soup did not improve the accuracy. Another option is to perform EMA, which has been shown effective in Block et al. (2023). We found that EMA was not helpful when applied to the 1k-step-interval checkpoints; more frequent averaging is likely required.

generative process naturally leads to the fact that multiple generations of a language model are more likely to contain a correct solution than a single one. Empirically, it has been widely observed that pass@k accuracy, namely, the accuracy by taking the best of k generations, is often much higher than pass@1. The main challenge is that without knowing the labels, the definition of "best" is usually unclear. A workaround is to apply some form of self-selection, such as by choosing the one with the highest logit or the most consistent solution (Wang et al., 2022; Li et al., 2022). There is, however, a notable limitation: generations can be consistent and confident yet inaccurate, making the self-consistency approach through majority voting less effective (Li et al., 2022).

Given these observations and inspired by findings in (Cobbe et al., 2021), we propose to use a separate verifier for selecting candidate generations. For each base generation SLM, we train a verifier to predict whether a generation is a correct solution to the given question. During inference, we generate multiple candidate generations using temperature sampling, and select the one with the highest verifier score.

Training data The training data consists of the SLM's generations on the labele GSM8K training set questions, paired with the binary labels indicating whether a generation leads to the correct numerical answer. We sample 48 generations for each training set question. The binary label for each generation is based on the final execution result and the ground truth label only, and we do not verify the correctness of intermediate steps. Note that this is the only time where the GSM8K training set is directly utilized in training.

Verfier model size	Base generation model size		
	125M	350M	1.3B
125M	68.9	68.8	71.7
350M	67.3	71.3	78.3
1.3B	76.1	79.2	**81.5**

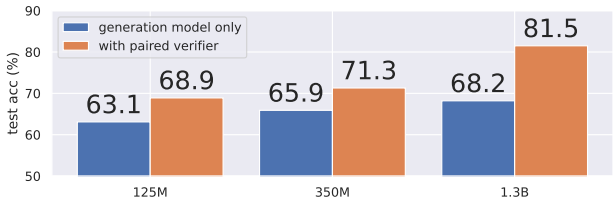

Figure 4: Pass@1 results on GSM8K test set with verifiers. For each test question, we sample 48 candidate answers from the base generation model, from which we submit the one with highest verifier score as the final answer. The verifier's score on a candidate answer is determined using its score on the last token.

Training setup The verifier is trained with a sequence-to-sequence task, where we use the binary label on the entire sequence to supervise each token. We find this approach improves consistently over training with a sequence classification task (i.e. only predicting a binary label on the entire sequence). The verifier model is initialized to be the same as the SLM, with an additional prediction head shared across all positions. All network parameters are updated during verifier training, which significantly outperforms alternatives where only the prediction head is updated, or where the network is trained from scratch.

Checkpoint and data diversity The diversity of the training data proves to be important, even if this may result in slightly sacrificing the data quality[7]: we find that including data generated using various temperatures and different checkpoints improves the performance of the verifier. In particular, the training data for the verifier is generated from checkpoints at 1k, 6k and 12k steps, and both training and testing data use a mixture of data generated with temperature 0.5 and 0.7. Compared to training with generations from a single checkpoint, the use of multiple checkpoints improves the performance from 78.9% to 81.5%.

Note also that in general, while we train one verifier for each generative model, verifiers transfer reasonably well across models. For example, our 1.3B model was able to reach 78.1% with a 350M verifier trained on generations from a 350M model.

[7]Worse quality data refers to generations from earlier checkpoints, or sampled with a higher temperature: we found that the accuracy for temperature=1 can be more than 10% worse than temperature=0 (i.e. greedy decoding).

Question:
Sophia is thinking of taking a road trip in her car, and would like to know how far she can drive on a single tank of gas. She has traveled 100 miles since last filling her tank, and she needed to put in 4 gallons of gas to fill it up again. The owner's manual for her car says that her tank holds 12 gallons of gas. How many miles can Sophia drive on a single tank of gas?

Generations:

1) correct:

```
distance_traveled = 100
gas_used = 4
tank_size = 12
miles_per_gallon= distance_traveled /
    gas_used
result = miles_per_gallon * tank_size
```

2) wrong:

```
distance_traveled = 100
gas_put_in = 4
gas_tank_size = 12
result = (distance_traveled
    + gas_put_in) / (gas_tank_size -
    gas_put_in)
```

3) correct:

```
miles_traveled = 100
gallons_added = 4
total_gallons = 12
miles_per_gallon = miles_traveled //
    gallons_added
total_miles = (total_gallons -
    gallons_added) * miles_per_gallon +
    miles_traveled
result = total_miles
```

Figure 5: Visualization of the verifier's token-level predictions. Text colors denote the verifier's prediction scores: correct, potentially correct, potentially wrong, and wrong. In all the three examples, the verifier's final prediction (on the last token) aligns with the groundtruth labels. In generation 1) and 2) the verifier's token-level scores appear to be interpretable and aligned with human assessment. However, in generation 3), the scores are rather strange. This suggests the verifier relies on special patterns of the model generations that may not be unversaly generalizable, even though its final predictions are fairly reliable.

Generation model size vs verifier size In Figure 4, we present results from a cross-examination of various generation model sizes + verifier model sizes. Interestingly, while the best accuracy is achieved with configuration with largest sizes, the verifier size seems to play a bigger role than the generation model size. The effect of model size scaling is surprisingly mild: as shown in Table 1, increasing the base generation model from 125M (Phi-1.5-tiny) to 1.3B (Phi-1.5) only gives a 6% boost. On the other hand, the verifier seems to be much more parameter efficient. For example, 125M generation model + 1.3B verifier can achieve 76.1%, while 1.3B generation model + 125M verifier gets only 71.7% Figure 4.

5 Robustness and decontamination

5.1 Contamination test

While we never use the GSM8K test set during training, TinyGSM consists entirely of synthetic data generated by GPT models, which may be contaminated since GPT-3.5-turbo may have been exposed to the test set during its own training, which would have led to some generated synthetic samples being replicating part of the test set. To prevent contamination, we decontaminate TinyGSM by checking for n-gram matches. We use $n = 13$ following standard practices (Brown et al., 2020; Wei et al., 2021; Du et al., 2022), [8] and remove punctuation and numbers before computing the matching. Out of the 11.0M unique synthetic questions [9] , 22 questions have a nonzero 13-gram match with the test set, and 38k questions (i.e. around 0.35% of the full set) have non-zero 8-gram matches. Examples of 13-gram matches are provided in Appendix A.2.

[8] n-gram matching is not sufficient for guarding against some other types of contamination (e.g. with respect to paraphrasing). However, we are not aware of better checks. One alternative is to check embedding similarity, though our clustering results on CodeGen 350M (Nijkamp et al., 2022) embeddings suggest that the embedding mostly reflects the semantic (topics) rather than structural/functional similarity, making it unfit for checking similarity in math questions. To our knowledge, state-of-the-art papers on training set contamination only test for exact matching (Shi et al., 2023b; Oren et al., 2023), and checking for contamination beyond exact match remains an open problem.

[9] The number of unique questions is smaller than the number of question-solution pairs since some questions were sampled more than once in the second step of the 2-step generation (Appendix A.1) and hence have multiple solutions.

5.2 Evaluation on SVAMP

Verfier model size	Base generation model size		
	125M	350M	1.3B
125M	63.2	70.0	72.2
350M	64.6	68.7	72.3
1.3B	74.1	79.0	75.6

Figure 6: SVAMP test accuracies.

For evaluating robustness of our models, we test on the SVAMP (Simple Variations on Arithmetic Math word Problems) dataset (Patel et al., 2021), consisting of 1000 math word problem questions with a focus on arithmetics. SVAMP constructed by applying certain types of variations to a set of base questions. Even though the base questions are generally considered easier than GSM8K [10], the variations may often confuse LLMs, thus making it a challenging benchmark for robustness. Our 1.3B model achieves 75.6% on SVAMP without further finetuning, indicating the robustness of the model.

6 Discussions

In this work, we showed a simple approach that enabled a 1.3B generation model to achieve 81.5% on the GSM8K dataset, setting a new state-of-the-art for small language models and raising the performance curve for scaling. Our approach consists of two simple steps: 1) collecting *TinyGSM*, a GPT-3.5 generated synthetic dataset which we will fully release, and 2) using a verifier that scores how likely a generation is correct, whose quality is boosted by utilizing diverse generations. Our results provide positive evidence that small language models have more potentials to be unlock and can be used for efficient. For future directions,

- *Leveraging different formats*: `TinyGSM` uses Python code as solutions, inspired by the observation that language models tend to struggle at calculations. However, we found that different solution formats, i.e. code versus natural language, can be complementary: while code helps circumvent errors related to execution or calculation, it tends to perform worse at questions that require equation solving, likely due to the fact that the Python syntax does not naturally support equations. Properly combining both formats has the potential to further boost performance.
- *The effect of verifier size*: Our results show that given a budget on the model size, scaling the verifier may be a more efficient use of the parameters. This counters our intuition that verification is an easier task than generation (which involves search), though there might be connections to findings in GAN training where the size of discriminator (Arora et al., 2018). Exploring the parameter efficiency in a generation model versus a verifier is an interesting future direction.

References

Sina Alemohammad, Josue Casco-Rodriguez, Lorenzo Luzi, Ahmed Imtiaz Humayun, Hossein Babaei, Daniel LeJeune, Ali Siahkoohi, and Richard G. Baraniuk. Self-consuming generative models go mad. *arXiv preprint arXiv: 2307.01850*, 2023.

Sanjeev Arora, Andrej Risteski, and Yi Zhang. Do GANs learn the distribution? some theory and empirics. In *International Conference on Learning Representations*, 2018. URL https://openreview.net/forum?id=BJehNfW0-.

Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Albert Q. Jiang, Jia Deng, Stella Biderman, and Sean Welleck. Llemma: An open language model for mathematics. *arXiv preprint arXiv: 2310.10631*, 2023.

[10] See Table 1 in Xie et al. (2023).

Adam Block, Dylan J. Foster, Akshay Krishnamurthy, Max Simchowitz, and Cyril Zhang. Butterfly effects of sgd noise: Error amplification in behavior cloning and autoregression. *arXiv preprint arXiv: 2310.11428*, 2023.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (eds.), *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html.

Stephanie C. Y. Chan, Adam Santoro, Andrew Kyle Lampinen, Jane X. Wang, Aaditya K Singh, Pierre H. Richemond, J. Mcclelland, and Felix Hill. Data distributional properties drive emergent in-context learning in transformers. *Neural Information Processing Systems*, 2022. doi: 10.48550/arXiv.2205.05055.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv: Arxiv-2110.14168*, 2021.

Nan Du, Yanping Huang, Andrew M. Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, Barret Zoph, Liam Fedus, Maarten P. Bosma, Zongwei Zhou, Tao Wang, Yu Emma Wang, Kellie Webster, Marie Pellat, Kevin Robinson, Kathleen S. Meier-Hellstern, Toju Duke, Lucas Dixon, Kun Zhang, Quoc V. Le, Yonghui Wu, Zhifeng Chen, and Claire Cui. Glam: Efficient scaling of language models with mixture-of-experts. In *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, volume 162 of *Proceedings of Machine Learning Research*, pp. 5547–5569. PMLR, 2022. URL https://proceedings.mlr.press/v162/du22c.html.

Benjamin L. Edelman, Surbhi Goel, Sham Kakade, Eran Malach, and Cyril Zhang. Pareto frontiers in neural feature learning: Data, compute, width, and luck. *arXiv preprint arXiv: 2309.03800*, 2023.

Ronen Eldan and Yuanzhi Li. Tinystories: How small can language models be and still speak coherent english? *arXiv preprint arXiv: Arxiv-2305.07759*, 2023.

Google Gemini Team. Gemini: A family of highly capable multimodal models.

Zhibin Gou, Zhihong Shao, Yeyun Gong, yelong shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. Tora: A tool-integrated reasoning agent for mathematical problem solving. *arXiv preprint arXiv: 2309.17452*, 2023.

Arnav Gudibande, Eric Wallace, Charlie Snell, Xinyang Geng, Hao Liu, Pieter Abbeel, Sergey Levine, and Dawn Song. The false promise of imitating proprietary llms. *arXiv preprint arXiv: 2305.15717*, 2023.

Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Harkirat Singh Behl, Xin Wang, Sébastien Bubeck, Ronen Eldan, Adam Tauman Kalai, Yin Tat Lee, and Yuanzhi Li. Textbooks are all you need. *arXiv preprint arXiv: 2306.11644*, 2023.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. *NeurIPS Datasets and Benchmarks*, 2021.

Gao Huang, Yixuan Li, Geoff Pleiss, Zhuang Liu, J. Hopcroft, and Kilian Q. Weinberger. Snapshot ensembles: Train 1, get m for free. *International Conference on Learning Representations*, 2017.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b. *arXiv preprint arXiv: 2310.06825*, 2023.

Rik Koncel-Kedziorski, Subhro Roy, Aida Amini, Nate Kushman, and Hannaneh Hajishirzi. Mawps: A math word problem repository. In *Proceedings of the 2016 conference of the north american chapter of the association for computational linguistics: human language technologies*, pp. 1152–1157, 2016.

S. Laine and Timo Aila. Temporal ensembling for semi-supervised learning. *International Conference on Learning Representations*, 2016.

Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Solving quantitative reasoning problems with language models, 2022.

Yifei Li, Zeqi Lin, Shizhuo Zhang, Qiang Fu, Bei Chen, Jian-Guang Lou, and Weizhu Chen. Making large language models better reasoners with step-aware verifier. *arXiv preprint arXiv: 2206.02336*, 2022.

Yuanzhi Li, Sébastien Bubeck, Ronen Eldan, Allie Del Giorno, Suriya Gunasekar, and Yin Tat Lee. Textbooks are all you need ii: **phi-1.5** technical report. *arXiv preprint arXiv:2309.05463*, 2023.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step, 2023.

Haipeng Luo, Qingfeng Sun, Can Xu, Pu Zhao, Jianguang Lou, Chongyang Tao, Xiubo Geng, Qingwei Lin, Shifeng Chen, and Dongmei Zhang. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. *arXiv preprint arXiv: 2308.09583*, 2023.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv: 2303.17651*, 2023.

Shen-Yun Miao, Chao-Chun Liang, and Keh-Yih Su. A diverse corpus for evaluating and developing english math word problem solvers. *Annual Meeting of the Association for Computational Linguistics*, 2020. doi: 10.18653/v1/2020.acl-main.92.

Seyed-Iman Mirzadeh, Mehrdad Farajtabar, Ang Li, Nir Levine, Akihiro Matsukawa, and Hassan Ghasemzadeh. Improved knowledge distillation via teacher assistant. *arXiv preprint arXiv: 1902.03393*, 2019.

Arindam Mitra, Luciano Del Corro, Shweti Mahajan, Andres Codas, Clarisse Simoes, Sahaj Agarwal, Xuxi Chen, Anastasia Razdaibiedina, Erik Jones, Kriti Aggarwal, Hamid Palangi, Guoqing Zheng, Corby Rosset, Hamed Khanpour, and Ahmed Awadallah. Orca 2: Teaching small language models how to reason, 2023.

Ansong Ni, Jeevana Priya Inala, Chenglong Wang, Alex Polozov, Christopher Meek, Dragomir Radev, and Jianfeng Gao. Learning math reasoning from self-sampled correct and partially-correct solutions. In *The Eleventh International Conference on Learning Representations*, 2023a. URL https://openreview.net/forum?id=4D4TSJE6-K.

Ansong Ni, Jeevana Priya Inala, Chenglong Wang, Alex Polozov, Christopher Meek, Dragomir Radev, and Jianfeng Gao. Learning math reasoning from self-sampled correct and partially-correct solutions. In *The Eleventh International Conference on Learning Representations*, 2023b. URL https://openreview.net/forum?id=4D4TSJE6-K.

Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv: 2203.13474*, 2022.

OpenAI. Gpt-4 technical report, 2023.

Yonatan Oren, Nicole Meister, Niladri Chatterji, Faisal Ladhak, and Tatsunori B. Hashimoto. Proving test set contamination in black box language models. *arXiv preprint arXiv: 2310.17623*, 2023.

Arkil Patel, S. Bhattamishra, and Navin Goyal. Are nlp models really able to solve simple math word problems? *North American Chapter Of The Association For Computational Linguistics*, 2021. doi: 10.18653/V1/2021.NAACL-MAIN.168.

Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, E. Chi, Nathanael Scharli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. *International Conference on Machine Learning*, 2023a. doi: 10.48550/arXiv.2302.00093.

Weijia Shi, Anirudh Ajith, Mengzhou Xia, Yangsibo Huang, Daogao Liu, Terra Blevins, Danqi Chen, and Luke Zettlemoyer. Detecting pretraining data from large language models. *arXiv preprint arXiv: 2310.16789*, 2023b.

Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Yarin Gal, Nicolas Papernot, and Ross Anderson. The curse of recursion: Training on generated data makes models forget. *arXiv preprint arXiv: 2305.17493*, 2023.

Ben Sorscher, Robert Geirhos, Shashank Shekhar, Surya Ganguli, and Ari Morcos. Beyond neural scaling laws: beating power law scaling via data pruning. *Advances in Neural Information Processing Systems*, 35:19523–19536, 2022.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv: 2307.09288*, 2023.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback. *arXiv preprint arXiv: 2211.14275*, 2022.

Xuezhi Wang, Jason Wei, D. Schuurmans, Quoc Le, E. Chi, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *International Conference on Learning Representations*, 2022. doi: 10.48550/arXiv.2203.11171.

Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi D. Q. Bui, Junnan Li, and Steven C. H. Hoi. Codet5+: Open code large language models for code understanding and generation. *arXiv preprint arXiv: 2305.07922*, 2023.

Jason Wei, Maarten Bosma, Vincent Zhao, Kelvin Guu, A. Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned language models are zero-shot learners. *International Conference on Learning Representations*, 2021.

Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, E. Chi, Tatsunori Hashimoto, Oriol Vinyals, P. Liang, J. Dean, and W. Fedus. Emergent abilities of large language models. *Trans. Mach. Learn. Res.*, 2022a. doi: 10.48550/arXiv.2206.07682.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, E. Chi, F. Xia, Quoc Le, and Denny Zhou. Chain of thought prompting elicits reasoning in large language models. *Neural Information Processing Systems*, 2022b.

Yixuan Weng, Minjun Zhu, Fei Xia, Bin Li, Shizhu He, Kang Liu, and Jun Zhao. Large language models are better reasoners with self-verification. *arXiv preprint arXiv: 2212.09561*, 2022.

Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In Kamalika Chaudhuri,

Stefanie Jegelka, Le Song, Csaba Szepesvári, Gang Niu, and Sivan Sabato (eds.), *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, volume 162 of *Proceedings of Machine Learning Research*, pp. 23965–23998. PMLR, 2022. URL https://proceedings.mlr.press/v162/wortsman22a.html.

Yuxi Xie, Kenji Kawaguchi, Yiran Zhao, Xu Zhao, Min-Yen Kan, Junxian He, and Qizhe Xie. Decomposition enhances reasoning via self-evaluation guided decoding. *arXiv preprint arXiv: 2305.00633*, 2023.

Fei Yu, Anningzhe Gao, and Benyou Wang. Outcome-supervised verifiers for planning in mathematical reasoning. *arXiv preprint arXiv: 2311.09724*, 2023a.

Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T. Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. Metamath: Bootstrap your own mathematical questions for large language models. *arXiv preprint arXiv: 2309.12284*, 2023b.

Xiang Yue, Xingwei Qu, Ge Zhang, Yao Fu, Wenhao Huang, Huan Sun, Yu Su, and Wenhu Chen. Mammoth: Building math generalist models through hybrid instruction tuning. *arXiv preprint arXiv: 2309.05653*, 2023.

Xu Zhao, Yuxi Xie, Kenji Kawaguchi, Junxian He, and Qizhe Xie. Automatic model selection with large language models for reasoning. *arXiv preprint arXiv: 2305.14333*, 2023.

Chuanyang Zheng, Zhengying Liu, Enze Xie, Zhenguo Li, and Yu Li. Progressive-hint prompting improves reasoning in large language models. *ARXIV.ORG*, 2023. doi: 10.48550/arXiv.2304.09797.

Aojun Zhou, Ke Wang, Zimu Lu, Weikang Shi, Sichun Luo, Zipeng Qin, Shaoqing Lu, Anya Jia, Linqi Song, Mingjie Zhan, and Hongsheng Li. Solving challenging math word problems using gpt-4 code interpreter with code-based self-verification. *arXiv preprint arXiv: 2308.07921*, 2023.

A Additional details on TinyGSM

A.1 Other prompts

The majority of the TinyGSM was generated using the prompt in Figure 3, where GPT-3.5-turbo is asked to generate both the question and the corresponding solution. The remaining data, including all data based on GSM-IC, is generated using a two-step process, where the first step prompts the model to generate question variants, and the second step asks to generate Python solutions given a question variant generated in the first step. The exact prompts are provided in Figure 7–Figure 9.

```
    Design 10 grade-school math problems similar to a given math problem.
    - Make sure the language is different and sufficiently rephrased.
    - Feel free to change the scenario, story and quantities.
    - Make sure the new problems are no easier than the given problem and require more steps to solve.
    - Make sure the new problems are self-complete, clear, unambiguous, and coherent.
    - Please do not include hints or solutions for the new problems.

    ## The original math problem

    {{question}}

    ## New problems

    - Problem 1:
```

Figure 7: The prompt template for generating question variants based on GSM8K.

```
    Please write 10 questions similar in style to a given question, where the new questions also contain
irrelevant information. Make sure to change the name and scenarios.

    # Original question

    {{question}}

    # New questions
    - Question 1:
```

Figure 8: The prompt template for generating question variants based on GSM8K-IC.

A.2 Contamination check: 13-gram collisions

There are 22 questions (out of 11.0M) with 13-gram collisions to test set questions. Examples are shown in Figure 10.

B Pretrained vs Random Init

In this section, we present a comparison of training on `TinyGSM` from a random initialization versus from a pretrained model.

Model size	125M	350M	1.3B
Random Init	53.1	55.5	57.3
pretrained	63.3	65.9	68.2

Table 2: Comparison of performance with and without pretraining.

```
    Please generate a detailed and complete Python program to solve a given math question.

    - Use variables to represent the quantities in the question.
    - Solve the question **step by step** (do NOT give the result directly, **DO NOT write calculations in the
comments**).
    - The program should contain multiple lines of code and end with 'result = XXX' (Make sure to replace XXX with
 the actual result of the python program!!!).
    - Then, the result should be printed out in the format of f'<<<{result}>>>'.
    - Make sure your Python program is complete and solves the problem. Do **NOT** write things like 'solution to
be completed', result = ?, insert your code here etc.
    - Give the complete solution to solve the problem in Python. Do not write things like 'insert your code here'.
    - You should solely rely on the Python program to solve the question. Do not do calculations in the comments.
The comment should not include any numbers.
    - Try to use fewer comments since they are expensive.
    - If you really want to solve equations like x = ..., try to use ``import sympy`` and **sympy.solve()**. sympy
.solve(expression) returns the roots of the expression. Do not write down calculations in the comments!
    - If you need to calculate the ceiling of a number, use `import math` then `math.ceil().`
    - End the Python program with <|endofprogram|> in a new line.

    ### Question

    {{question}}

    ### Program
```

Figure 9: The prompt template for generating code solution for a given question.

```
    # Q: Daniel has   brothers His older brother is   years older than   times Daniel's age when Daniel was  years
 old His younger brother is   years old which is  the age of the older brother What is their combined age

    match:
    In a family there are   brothers and   sisters All sisters are the same age which is   One of the brothers is
 **years old which is  the age of the older brother What is** the total age of all these siblings

    # Q: Two cars are driving on a highway The first car is traveling at an average speed of   miles per hour
while the second car takes a minute break after driving for   minutes how long can they remain stopped before the
first car catches up with them

    match:
    **Two cars are driving on a highway The first car is traveling at an average speed of   miles per hour** when
the second car passes it at an average speed of  miles per hour  If both cars continue on the highway at the same
speed how many miles will separate them after  hours

    # Q: Leo and Nora sold lemonade at a stand Leo sold  cups at  cents each and Nora sold  cups at  cents each
They decided to split the money equally How much money did each of them get

    match:
    While walking down the street with his  young siblings Greg found  To be fair to his siblings he **decided to
split the money equally How much money did each of them get**


    # Q: A bookstore is selling a book for  while is a  discount from the original price What was the original
price of the book

    match:
    Kyle bought last year's bestselling book for  This is with a **discount from the original price What was the
original price of the book**
```

Figure 10: The prompt template for generating code solution for a given question.